

Programa · Comisión 2026

AI Engineering

Clase 6:

Sistemas multi-agente: colaboración y especialización

Objetivo

1. Diferenciar entre topologías jerárquicas y cooperativas en el diseño de sistemas multi-agente.
2. Implementar el Patrón Supervisor para delegación dinámica utilizando LangGraph.
3. Gestionar estado compartido y comunicación asíncrona validada con Pydantic.
4. Diseñar mecanismos de validación determinística y resolución de conflictos entre agentes.
5. Evaluar cuándo escalar a revisión humana ante fallos de validación irresolubles.



Agend

1. Topologías multi-agente: colaboración vs. jerarquía.
2. El Patrón Supervisor y el estado compartido entre agentes.
3. Validación y resolución de conflictos entre agentes.
4. Pre-Entrega 6: tu orquestador multi-agente especializado.

a

**¡Esta clase va a ser
grabada!**



01

Topologías Multi-Agente

Del Agente al Sistema

Cuándo un solo prompt no alcanza

Cuando un problema es demasiado grande para un solo prompt o bucle de razonamiento, la complejidad cognitiva desborda a un único agente.

La topología multi-agente define quién tiene la autoridad, cómo se distribuye el estado y qué canales de comunicación existen.

No es solo un diagrama: determina cómo escalará tu sistema bajo carga y qué tan fácil será depurarlo.

Topología Jerárquica

El modelo del Supervisor

- **Delegación:** el usuario envía la petición a un Supervisor, que la descompone en subtarear.
- **Asignación:** el Supervisor llama a agentes específicos (Búsqueda, Redacción, Validación) y agrega sus resultados.
- **Ejemplo:** una Agencia de Viajes Autónoma. El Planner coordina vuelos, hoteles e itinerario sin que los especialistas se enteren de los problemas del otro.

Los especialistas nunca hablan entre sí: todo pasa por el Supervisor.

Jerarquía: Ventajas y Riesgo

Orden a cambio de un cuello de botella

Control Centralizado: es fácil predecir el flujo y auditar decisiones en un solo punto.

Modularidad: podés cambiar un agente especialista sin tocar el resto del sistema.

El gran riesgo: el supervisor es un cuello de botella. Si falla o se confunde, todo el sistema queda paralizado.

Topología Cooperativa

Pares que colaboran, sin jefe

Sin jefe: los agentes son pares que se comunican entre sí; la coordinación surge de un estado compartido.

Pizarrón (Blackboard): todos escriben en un espacio compartido; si un agente ve algo que puede resolver, actúa sin que se lo pidan.

Coreografía: cada agente sabe qué hacer cuando recibe un mensaje de otro, creando una reacción en cadena.

Ejemplo: el Arquitecto escribe la especificación, el Programador la codifica, el Tester la prueba, y el ciclo se repite si hay errores.

Cooperativa: Ventajas y Riesgo

Resiliencia

No hay un punto único de fallo.
Si un agente se cae, otros
pueden seguir trabajando.

Escalabilidad Orgánica

Es más fácil añadir agentes que
escuchen eventos, sin
reconfigurar un orquestador
central.

El caos emergente

Sin supervisión, los agentes
pueden entrar en bucles
infinitos o producir resultados
inconsistentes.

¿Cuál Elegir?

La topología depende del problema

Jerarquía: workflows deterministas, cuando la seguridad es crítica, o cuando combinás un modelo grande (supervisor) con modelos chicos (workers).

Colaboración: problemas explorativos, trabajo en paralelo masivo sobre un mismo dataset, o sistemas de tiempo real donde un supervisor central sería demasiado lento.

No hay una topología "mejor": hay una más simple que resuelve tu problema con la menor latencia y costo posibles.

Momentos de la Clase:

Actividad colaborativa

Para pensar

Para recordar

Coder Tips

Pensá en un equipo humano que conozcas bien. ¿Funciona más como una jerarquía con un jefe, o como pares que colaboran sin uno? ¿Por qué creés que se organizó así?

Práctica - Unidad 1

Diseña tu topología

Objetivo: diseñar una topología jerárquica para un sistema de atención al cliente en e-commerce.

Manos a la obra:

- Definí al menos 3 agentes especialistas y un supervisor, con sus responsabilidades y herramientas.
- Describí el flujo lógico: desde que llega el Tweet hasta que se envía la respuesta.

Resultado: un documento con la propuesta arquitectónica y la justificación de por qué elegiste jerarquía.

02

Patrón Supervisor y Estado

Patrón Supervisor

El Editor Jefe de tu sistema

Analogía: una redacción de noticias. Un Editor Jefe (Supervisor) recibe la noticia, decide quién investiga, revisa el borrador y aprueba la publicación.

En LangGraph, el Supervisor es un nodo central que analiza el estado, toma una decisión de ruteo y valida el progreso.

Puede pedirle a un agente que repita su tarea si el resultado no es satisfactorio.

Por Qué Delegar

Cadena Simple

Un solo flujo lineal intenta hacer todo: no escala ni se adapta a tareas complejas.

Patrón Supervisor

Especialización de modelos, escalabilidad y recuperación de errores, todo en un mismo diseño.

El resultado

Sumar una funcionalidad nueva es tan simple como crear un nodo y registrarlo en el supervisor.

Implementación en LangGraph

Aristas condicionales

El Supervisor no tiene una ruta fija: depende de Aristas Condicionales que leen su decisión de ruteo. Analiza el estado actual (el historial de mensajes) para entender qué se hizo hasta ahora.

Un error común: que el Supervisor haga el trabajo de los especialistas. El Supervisor coordina, el especialista ejecuta.

De Diccionarios a Esquemas

El estado como contrato

En unidades anteriores usamos diccionarios simples para el estado compartido entre agentes.

- **En un sistema de grado industrial necesitamos contratos: validar con Pydantic antes de inyectar datos al estado.**
- **Aislamiento de Escritura: cada agente es dueño de una parcela del estado, por ejemplo `state["research_results"]`.**
- **Si el LLM alucina un formato de fecha incorrecto o un score fuera de rango, el error se captura inmediatamente.**

Esto evita condiciones de carrera donde un agente sobrescribe lo que otro acaba de calcular.

Por Qué Asyncio es Vital

Los LLMs son lentos, tu sistema no

Los LLMs son intrínsecamente lentos: una llamada puede tardar varios segundos.

Con 5 agentes síncronos, el tiempo total de respuesta es la suma de todos los retardos.

- Con asyncio, mientras un agente espera una respuesta, el orquestador puede procesar otros resultados o escribir en la base vectorial.

Consistencia Eventual: un nodo solo se ejecuta cuando sus dependencias completaron su escritura.

Errores del Estado

El estado global es una trampa

Shared Mutable State: usar una lista global de Python con `.append()` desde múltiples funciones async falla bajo alta concurrencia.

No manejar excepciones en nodos: un nodo asíncrono que falla sin capturarse deja el estado del sistema bloqueado.

Usá siempre el mecanismo de retorno de estados del grafo, nunca variables globales compartidas.

Errores del Supervisor

Tres trampas de diseño

El Supervisor Todólogo: un supervisor que intenta hacer parte del trabajo de los agentes infla el prompt y gasta tokens.

Falta de Contratos de Interfaz: no definir qué forma tiene el dato que un agente pasa al siguiente; en LangGraph esto se gestiona con el State.

El Bucle Infinito de Corrección: un Crítico que siempre encuentra algo mal y un Refinador que siempre intenta arreglarlo, sin fin.

Momentos de la Clase:

Para recordar

Coder Tips

Material ampliado

Cuestionario de tarea

#FindTheBug

La palabra clave del día es
Aislamiento: cada agente
escribe solo en su parcela del
estado. Si dos agentes pueden
pisarse los datos, tu arquitectura
tiene una condición de carrera
esperando pasar.

Práctica - Unidad 2

Construí tu Supervisor

Objetivo: implementar un Supervisor que delega dinámicamente entre un Analista y un Escritor.

Manos a la obra:

- Definí AgentState con messages y next; el Supervisor usa with_structured_output() para elegir el próximo nodo.
- Conectá los especialistas de vuelta al supervisor, y el supervisor a ellos con add_conditional_edges.

Resultado: un grafo que delega, valida y decide cuándo finalizar (END), ejecutado con graph.astream().

Práctica - Unidad 2b

Estado validado y asíncrono

Objetivo: validar el estado compartido entre agentes con Pydantic, de forma asíncrona.

Manos a la obra:

- Definí ResearchArtifact y ReviewArtifact con Pydantic, incluyendo un confidence_score entre 0 y 1.
- Hací que cada nodo async escriba solo en su propia sección del estado (researcher_output, reviewer_output).

Resultado: un grafo donde el investigador y el revisor nunca pisan los datos del otro.

Break,

andá por un cafecito
o mate, ¡y volvemos!



03

Validación y Conflictos

Validación Determinística

Pydantic, no vibras

Un agente que extrae fechas financieras puede alucinar un 31 de noviembre: un error de lógica, no de creatividad.

La solución más robusta no es "pedirle al modelo que revise", sino un validador determinístico con Pydantic o Python.

Que un texto "suene profesional" no garantiza que sea correcto: un modelo puede ser muy persuasivo y estar equivocado.

El Árbitro

Trazabilidad

Verificar qué fuentes respaldan cada afirmación en conflicto: de dónde viene el dato importa.

Agente Árbitro

Un agente especializado en detectar contradicciones y pedir evidencia a los implicados.

Ni Promedio ni Mayoría

Dos agentes pueden alucinar lo mismo si comparten modelo base: el consenso no garantiza la verdad.

El Bucle de Reparación

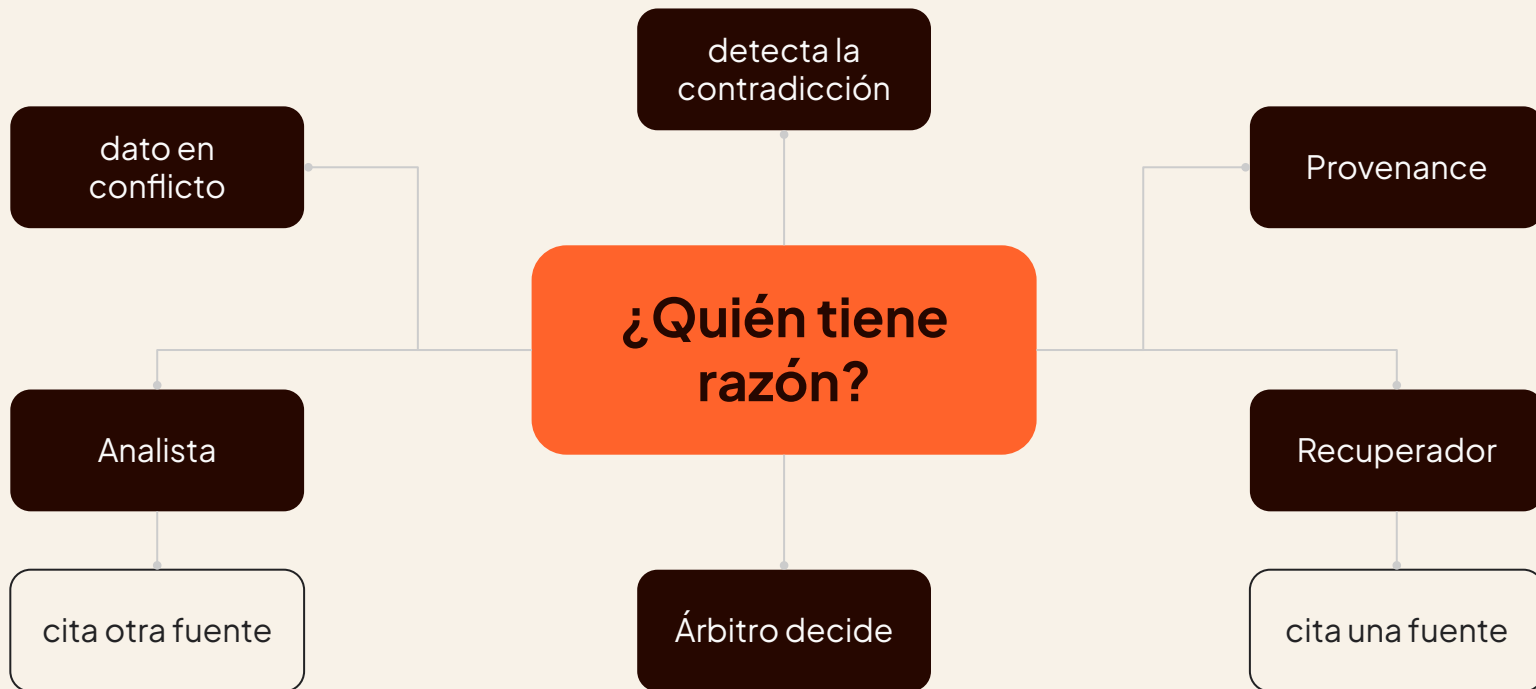
Error específico, no un reintento ciego

Cuando una validación falla, no le pidas al agente que "intente de nuevo": es probable que repita el mismo error.

Reenvía el output fallido junto con el mensaje de error específico del validador, para que corrija solo esa parte.

Estado Mínimo: pasá solo el contexto necesario entre nodos. Todo el historial diluye las instrucciones de validación.

Un error específico da información de ingeniería; un prompt idéntico solo repite el mismo sesgo.



Práctica - Unidad 3

Diseña tu puerta de calidad

Objetivo: diseñar una puerta de calidad para un sistema donde el Analista contradice al Recuperador.

Manos a la obra:

- Definí qué validación determinística aplicarías antes de aceptar cualquier dato numérico o de fecha.
- Diseña el prompt de reparación que le enviarías al agente que falló, incluyendo el error específico.

Resultado: un criterio claro de cuándo escalar a revisión humana en vez de seguir reintentando.

PE

Pre-Entrega 6

¿Qué Debes Construir?

Pre-Entrega 6

Un Orquestador Multi-Agente de Análisis e Investigación con al menos 2 dominios de especialización y una síntesis final.

- **Topología Jerárquica:** un nodo Supervisor que actúa como router inteligente y controlador de flujo.
- **Mínimo 2 agentes especialistas (Búsqueda/Investigación y Análisis/Cómputo) y un estado compartido estructurado.**

Tu Misión

El entregable

 **Entrega: repo de GitHub con el orquestador, README con la topología elegida y un video o notebook del flujo**

 **El supervisor debe decidir si la tarea está completa o si un especialista debe refinar su output**

Construí un sistema donde puedas ver, en un log, exactamente quién decidió qué y por qué.

Pasos

1. Definí tu Estado: una clase TypedDict que herede de MessagesState, con campos como next_agent.
2. Creá los agentes especialistas con create_react_agent o prompts específicos, cada uno con herramientas acotadas.
3. Implementá el Supervisor: su prompt debe responder "¿quién debe intervenir ahora, o es momento de finalizar?".
4. Construí el grafo: add_node para cada agente y para el supervisor, con Conditional Edges entre ellos.
5. Probá la interacción: una consulta que fuerce al supervisor a delegar, recibir el dato, y cerrar la conversación.

Errores a Evitar

Antes de entregar, revisá esto

El Supervisor Infinito: no definir una condición de parada clara. Implementá un contador de pasos o un criterio de suficiencia.

- Contaminación de Contexto: pasar todo el historial a todos los agentes en cada turno.
- Cada especialista debería recibir solo la instrucción específica y el contexto necesario, no toda la metadata del sistema.
- No versionar los prompts del supervisor: un cambio pequeño puede romper todo el ruteo.

Olvidar el diagrama del grafo en el README: `draw_mermaid_png()` lo genera en segundos.

No probar el camino de fallo: simulá que un especialista devuelve un error y verificá que el supervisor no se rompa.

¿Cómo se evalúa?

· 100 pts

Documentación · 20%

Claridad en la explicación de la topología y representación visual del grafo en el README.

Arquitectura del Grafo · 35%

Diseño del StateGraph y la implementación del nodo Supervisor para la delegación dinámica.

Especialización y Estado · 45%

Agentes con herramientas funcionales, y uso de state.py con mecanismos de validación.

Síntesis del Módulo

De cadenas a organizaciones digitales

Pasamos de cadenas lineales a sistemas donde múltiples agentes se coordinan, se especializan y se validan entre sí.

- **La Jerarquía da orden, seguridad y trazabilidad; la Colaboración da flexibilidad, velocidad y resiliencia.**
- **El Patrón Supervisor y el estado validado con Pydantic son la diferencia entre un prototipo y un sistema confiable.**

Del cliente LLM robusto de la Clase 1 a un sistema multi-agente que razona y se corrige a sí mismo.

Especialización en Acción

Sesión en Vivo 6

Desafío "Agente Colectivo": simularán roles de agentes (Investigador, Analista, Escritor, Crítico) para resolver un puzzle de recursos limitados.

- **Clínica de Arquitectura:** revisión en vivo de sus diseños para identificar cuellos de botella en la comunicación.
- **Debate:** Autonomía vs. Control, ¿cuánta libertad puede tener un agente antes de volverse impredecible?

Traé tu Pre-Entrega 6 y el diagrama de tu arquitectura: los vamos a usar como base de la clínica.

¿Preguntas?

De un agente a una organización digital. Soltá tus dudas del módulo.



Glosario

1. Topología Multi-Agente: la estructura de red que define autoridad, estado y comunicación entre agentes.
2. Supervisor: agente que coordina, delega tareas y valida el progreso de los especialistas.
3. Worker / Agente Especialista: agente enfocado en una única tarea o dominio dentro del sistema.
4. Blackboard: patrón donde los agentes leen y escriben en un espacio de estado compartido.
5. Coreografía: coordinación donde cada agente reacciona a eventos, sin un director central.
6. Peer-to-Peer: topología donde los agentes se comunican como pares, sin jerarquía.

1. Aristas Condicionales: conexiones en un grafo cuyo destino depende del estado, no es fijo.
2. StateGraph: la estructura de LangGraph que define nodos, aristas y el estado compartido.
3. TypedDict / MessagesState: estructuras que definen la forma del estado que fluye entre nodos.
4. Aislamiento de Escritura: cada agente escribe solo en su propia sección del estado compartido.
5. Consistencia Eventual: aceptar que, por un instante, distintos agentes vean versiones ligeramente distintas del estado.
6. Shared Mutable State: antipatrón de usar variables globales mutables entre funciones asíncronas.

1. Validación Determinística: verificar una regla de negocio con código, no con el juicio del modelo.
2. Bucle de Reparación: reenviar un error específico al agente para que corrija su propia salida.
3. Provenance / Trazabilidad: rastrear qué fuente respalda cada afirmación, para resolver contradicciones.
4. Critic-Refiner: patrón donde un agente crítico evalúa y otro refina la salida, con una condición de parada.
5. Human-in-the-loop: escalar a revisión humana cuando la validación automática falla repetidamente.
6. Contaminación de Contexto: pasar más historial del necesario, diluyendo las instrucciones del agente.

¡Gracias!